فاین‌تیون ParsBERT برای دسته‌بندی اخبار فارسی

گزارش کارهای انجام‌شده، نتیجه‌ها و درس‌ها — مهر ۱۴۰۵ — کد در شاخهٔ `claude/loving-einstein-jh2u6g` ِ ریپوی `parsajafarpoor/ParsBERT-`

89.7%	6	3,000	17%
بهترین دقت روی val	کلاس خبری	خبر آموزشی	دقت حدس تصادفی

۱. هدف

هدف این بود که مدل زبانی فارسی **ParsBERT** را طوری آموزش دهیم (فاین‌تیون کنیم) که خلاصهٔ یک خبر فارسی را بخواند و دستهٔ آن را تشخیص دهد: **اقتصاد، بین‌الملل، ورزش، علم و فناوری، سیاست، سلامت**. خروجی نهایی فایل `predictions.csv` با ستون‌های `id,label` برای ۱۲۰۰ خبر بدون برچسب است.

۲. داده

داده زیرمجموعه‌ای متعادل از دیتاست **PN-Summary** (تیم Hooshvare، مجوز MIT) است. هر نمونه خلاصهٔ یک یا دو جمله‌ای از یک خبر است.

فایل	تعداد	برچسب	کاربرد
train.csv	3,000	دارد (۵۰۰ از هر کلاس)	آموزش مدل
val.csv	600	دارد (۱۰۰ از هر کلاس)	امتحان حین آموزش و انتخاب بهترین مدل
test.csv	1,200	ندارد	پیش‌بینی نهایی

بررسی اولیهٔ داده (قبل از نوشتن کد)

- **کلاس‌ها کاملاً متعادل‌اند**؛ پس دقت (accuracy) معیار قابل اعتمادی است.
- **متن‌ها کوتاه‌اند**: میانگین حدود ۲۷ کلمه و حداکثر ۱۴۲ کلمه؛ پس طول ۱۲۸ توکن کافی است.
- **داده تمیز است**: بدون مقدار خالی، بدون ردیف تکراری، بدون «ي/ك» ِ عربی، و همهٔ عددها فارسی‌اند. پس یکسان‌سازی حروف لازم نبود.
- **نیم‌فاصله (ZWNJ) زیاد است** (مثل «بین‌الملل»)؛ دلیل انتخاب نسخهٔ `bert-fa-zwnj-base` .

۳. روش: فاین‌تیون چیست؟

ParsBERT یک شبکهٔ عصبی Transformer با حدود ۱۱۰ میلیون وزن و ۱۲ لایه است که از قبل روی حجم زیادی متن فارسی با بازی «کلمهٔ جاافتاده را حدس بزن» آموزش دیده و زبان فارسی را بلد است. در فاین‌تیون:

1. **بدنهٔ مدل** (لایه‌های فارسی‌دان) نگه داشته می‌شود.

2. **سرِ قدیمی** (حدس کلمه، `cls.predictions`) کنار گذاشته می‌شود.
3. یک **سرِ دسته‌بند تازه** با ۶ خروجی (`classifier`) اضافه و همراه بدنه با دادهٔ ما آموزش داده می‌شود.

به این کار **یادگیری انتقالی** می‌گویند: با فقط ۳۰۰۰ نمونه نمی‌شد یک شبکه را از صفر فارسی و دسته‌بندی یاد داد، ولی مدلی که فارسی بلد است فقط باید دسته‌بندی یاد بگیرد. هشدارهای `UNEXPECTED / MISSING` هنگام بارگذاری دقیقاً همین تعویض سر را نشان می‌دهند و طبیعی‌اند.

۴. مراحل اسکریپت train.py

1. **خواندن داده** با pandas.
2. **تبدیل برچسب به عدد** (مثلاً «اقتصاد» ← 0) چون مدل فقط عدد می‌فهمد.
3. **توکن کردن:** هر جمله به تکه‌ها (توکن) و سپس شمارهٔ آن‌ها تبدیل می‌شود؛ `[CLS]` و `[SEP]` به ابتدا و انتهای جمله اضافه می‌شوند.
4. **ساخت مدل** با `AutoModelForSequenceClassification` و ۶ خروجی.
5. **آموزش** با Hugging Face ِ`Trainer` و ارزیابی روی val بعد از هر epoch؛ بهترین epoch نگه داشته می‌شود.
6. **گزارش:** دقت هر کلاس، ماتریس درهم‌ریختگی، و فایل `val_errors.csv` شامل خبرهای اشتباه.
7. **پیش‌بینی test** و ساخت `predictions.csv` ؛ امتیازهای خام (logits) هم برای ترکیب‌های بعدی ذخیره می‌شوند.

تنظیمات پیش‌فرض و دلیل هرکدام

تنظیم	مقدار	چرا
مدل پایه	bert-fa-zwnj-base	نیم‌فاصله را درست توکن می‌کند
epochs	3	آزمایش نشان داد ۵ epoch بهتر نیست
learning rate	2e-5	کوچک، تا دانش قبلی مدل خراب نشود
warmup	10%	نرخ یادگیری آرام بالا می‌رود تا سرِ تصادفیِ تازه، بدنه را به‌هم نریزد
weight decay	0.01	جلوگیری از حفظ کردن داده (overfitting)
batch size	16	مناسب برای حافظهٔ ۴ گیگ کارت گرافیک
max length	128	تقریباً همهٔ خبرها را پوشش می‌دهد
padding	dynamic	هر batch فقط تا طول بلندترین جملهٔ خودش پر می‌شود؛ سرعت بیشتر
fp16	GPU only	سرعت بیشتر روی GPU؛ با `--no_fp16` خاموش می‌شود
seed	42	تکرارپذیری نتیجه‌ها

۵. محیط اجرا

- **Colab** در ابتدا در دسترس نبود و Hugging Face از محیط توسعه مسدود بود؛ آموزش روی کامپیوتر شخصی انجام شد.

- **CPU**: هر آموزش ۳ epoch حدود ۳۰ دقیقه (حدود ۳ ثانیه برای هر قدم).
- **GPU** (GeForce GTX 1650، ۴ گیگ): نسخهٔ CUDA ِ کتابخانهٔ `2.6.0+cu124`) torch) به‌صورت دستی دانلود و در یک **محیط مجازی روی درایو D** نصب شد تا جای درایو C پر نشود. هنگام نصب بقیهٔ کتابخانه‌ها، نسخهٔ torch قفل شد تا pip آن را با نسخهٔ CPU عوض نکند.

۶. نتیجه‌ها

اجرا	سخت‌افزار	epochs	lr	دقت val	خطا
۱	CPU	3	2e-5	88.33%	70
۲	CPU	5	3e-5	88.00%	72
۳	GPU (fp16)	3	2e-5	**89.67%**	**62**

دقت هر کلاس در بهترین اجرا (اجرای ۳)

کلاس	precision	recall	f1
ورزش	0.990	0.990	0.990
سلامت	0.890	0.970	0.928
اقتصاد	0.888	0.870	0.879
بین‌الملل	0.861	0.870	0.866
علم و فناوری	0.861	0.870	0.866
سیاست	0.890	0.810	0.848

precision: از خبرهایی که مدل گفت متعلق به این دسته‌اند، چند درصد درست بود. **recall**: از خبرهای واقعی این دسته، چند درصد پیدا شد. **f1**: میانگین متعادل این دو.

ماتریس درهم‌ریختگی (اجرای ۳) — سطر = واقعی، ستون = پیش‌بینی

	اقتصاد	بین‌الملل	سلامت	سیاست	علم	ورزش
اقتصاد	**87**	2	2	2	**7**	0
بین‌الملل	2	**87**	0	**7**	4	0
سلامت	1	0	**97**	0	2	0
سیاست	6	**11**	1	**81**	1	0
علم و فناوری	1	1	**9**	1	**87**	1
ورزش	1	0	0	0	0	**99**

۷. تحلیل خطا

سه جفت کلاس بیشترین اشتباه را دارند:

- **سیاست ↔ بین‌الملل** (۱۱ + ۷ اشتباه، دوطرفه): خبرهای وزارت خارجه، آمریکا، تحریم و برجام در هر دو دسته دیده می‌شوند.
- **علم و فناوری ← سلامت** (۹): خبرهای پژوهش پزشکی، واکسن و کرونا.
- **اقتصاد ← علم و فناوری** (۷): خبرهای اینترنت، اپراتورها و استارتاپ‌ها.

چرا سیاست و بین‌الملل قاطی می‌شوند؟

شمارش کلمه‌ها در ۵۰۰ خبر هر دسته در train:

کلمه	سیاست	بین‌الملل	نتیجه
مجلس	166	5	نشانهٔ روشن سیاست
عراق / روسیه	10 / 10	41 / 40	نشانهٔ روشن بین‌الملل
آمریکا	71	118	در هر دو زیاد
تحریم	30	25	در هر دو تقریباً برابر

> **یافتهٔ اصلی — نویز برچسب:** نمونه‌هایی در train هست که کاملاً دربارهٔ آمریکا و چین‌اند ولی برچسب «سیاست» دارند، و نمونه‌هایی دربارهٔ ایران که «بین‌الملل» خورده‌اند. PN-Summary از چند خبرگزاری جمع شده و برچسب بیشتر به **بخشی از خبرگزاری که خبر را منتشر کرده** بستگی دارد تا به محتوای متن. این اطلاعات در متن نیست؛ پس بخشی از این اشتباه‌ها را هیچ مدلی نمی‌تواند رفع کند.

۸. آزمایش‌هایی که کمک نکردند

آزمایش	ایده	نتیجه روی val
۵ epoch و lr بزرگ‌تر	آموزش بیشتر	88.33% ← 88.00% **(بدون بهبود)**
balance.py	تعداد پیش‌بینی هر کلاس برابر شود (الگوریتم Sinkhorn)، چون مدل روی test بیشتر «بین‌الملل» می‌گفت	89.67% ← 89.00% **(ضرر)**
knn_fix.py	برای خبرهای تقریباً تکراری، برچسب نزدیک‌ترین خبر train گرفته شود (روی val، همهٔ ۳۳ خبر با شباهت ۰٫۴ یا بیشتر هم‌برچسب نزدیک‌ترین همسایه بودند)	89.67% ← 89.67% (BERT همه را از قبل درست گفته بود)

این آزمایش‌ها نشان دادند خبرهای مرزی واقعاً «شیر یا خط»اند: هل دادن آن‌ها به یک سمت به همان اندازه که درست می‌کند، خراب هم می‌کند. پس مشکل اصلی مدل نیست، **داده** است.

۹. درس‌ها

- **اول داده را ببین، بعد کد بنویس.** بررسی داده نشان داد یکسان‌سازی حروف بی‌فایده است و نیم‌فاصله مهم است.
- **هیچ ترفندی را بدون اندازه‌گیری قبول نکن.** دو ایدهٔ منطقی (balance و knn) روی val کمک نکردند و یکی ضرر هم زد.
- **نوسان تصادفی بزرگ است.** اجرای ۱ و ۳ تنظیمات یکسان داشتند و فقط سخت‌افزار و نسخهٔ کتابخانه فرق داشت، ولی ۱٫۳ درصد اختلاف داشتند. با ۶۰۰ نمونهٔ val، اختلاف‌های زیر حدود ۱٫۵٪ قابل اتکا نیستند.
- **هر بار فقط یک چیز را عوض کن** تا اثرش معلوم شود (در اجرای ۲ دو چیز هم‌زمان عوض شد).
- **سقف را داده تعیین می‌کند.** با این برچسب‌ها، دقت بالای حدود ۹۰ تا ۹۲ درصد بعید است.

۱۰. قدم‌های بعدی

کار	توضیح	وضعیت
ترکیب ۵ مدل (Ensemble)	آموزش با seed های ۱ تا ۵ روی Colab و میانگین احتمال‌ها با `ensemble.py`؛ اثر شانس را خنثی می‌کند	در حال اجرا
آموزش روی train+val	گزینهٔ `--train_on_all`؛ ۲۰٪ دادهٔ بیشتر برای مدل نهایی	بعد از ensemble
TAPT	ادامهٔ پیش‌آموزش MLM روی متن خبرها پیش از دسته‌بندی	اختیاری
پاک‌سازی برچسب	یافتن برچسب‌های احتمالاً اشتباه train با k-fold (confident learning)	اختیاری

۱۱. فایل‌های پروژه

فایل	کار
`train.py`	آموزش، ارزیابی، ماتریس درهم‌ریختگی، ساخت predictions.csv
`ensemble.py`	ترکیب چند مدل
`balance.py`	متعادل‌سازی پیش‌بینی‌ها (روی این داده کمک نکرد)
`knn_fix.py`	اصلاح با خبرهای تقریباً تکراری (روی این داده اثری نداشت)
`tokenizer_demo.py`	نمایش خروجی توکنایزر روی جمله‌ها
`data/`	train، val، test و راهنمای دیتاست

```
python train.py                                # one model
python train.py --seed 2 --output_dir outputs_s2  # another seed
python ensemble.py outputs_s1 outputs_s2 ...      # combine
python train.py --train_on_all                 # final model on train+val
```